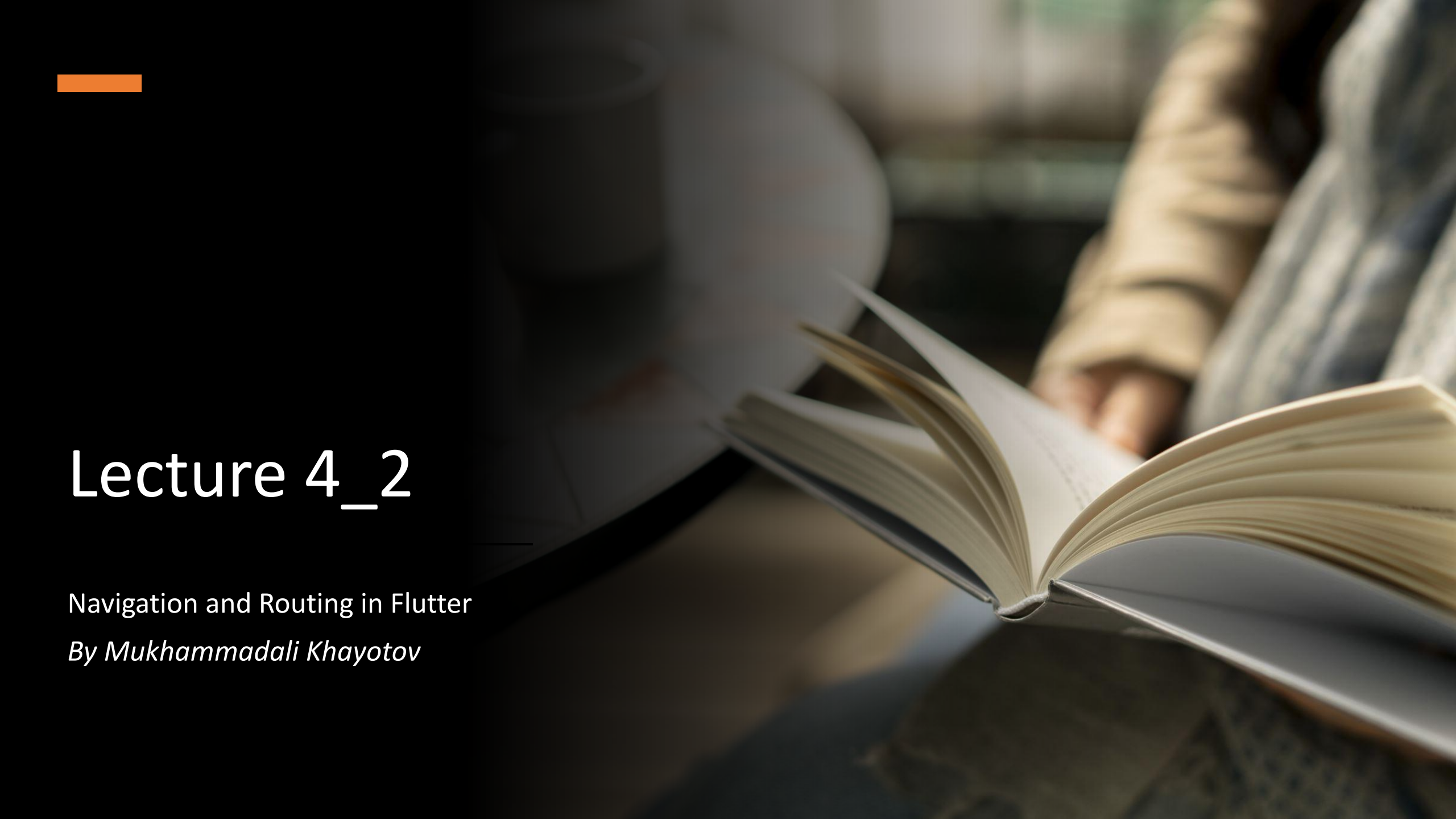


Lecture 4_2

Navigation and Routing in Flutter

By Mukhammadali Khayotov

Contents

StateFul Widgets (setState())

Introduction to Navigation and Routing in Flutter

- Importance of navigation in mobile apps
- Overview of navigation in Flutter

Navigating between screens

- The Navigator class and its methods (push and pop)
- Creating simple transitions between screens

Handling Routes in Flutter

- Definition of Routes
- Navigating by route names

Tab-based Navigation

- Using TabController class
- Customizing Tabs

Bottom Navigation bars

- Using BottomNavigationBar widget
- Handling interactions with the bottom navigation bar

Passing Data Between Screens using Route arguments

- Passing some data to a new screen
- Returning data from a screen

More About Navigations

- Nested Navigation
- Handling complex routing scenarios

setState() (example code)

```
22  class MyHomePage extends StatefulWidget {
23    const MyHomePage({super.key, required this.title});
24
25    final String title;
26
27    @override
28    State<MyHomePage> createState() => _MyHomePageState();
29  }
30
31  class _MyHomePageState extends State<MyHomePage> {
32    int _counter = 0;
33
34    void _incrementCounter() {
35      setState(() {
36        _counter++;
37      });
38    }
```

In this example, MyHomePage is a StatefulWidget that creates an instance of _MyHomePageState. _MyHomePageState is a class that extends State and includes a _counter variable and a _incrementCounter method. The build method returns a widget tree that includes a Scaffold with an AppBar, a Center with a Column of Text widgets, and a FloatingActionButton. The FloatingActionButton's onPressed property is set to _incrementCounter, which increments _counter and calls setState to trigger a rebuild.

setState() (example code)

```dart
  @override
  Widget build(BuildContext context) {

    return Scaffold(
      appBar: AppBar(
        backgroundColor: Theme.of(context).colorScheme.inversePrimary,
        title: Text(widget.title),
      ),  // AppBar
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            const Text(
              'You have pushed the button this many times:',
            ),  // Text
            Text(
              '$_counter',
              style: Theme.of(context).textTheme.headlineMedium,
            ),  // Text
          ],  // <Widget>[]
        ),  // Column
```

It is possible to use setState() in multilple locations to invoke rebuild for different widget updates.

setState() (example code)

```
52                const Text(
53                  'You have pushed the button this many times:',
54                ),  // Text
55                Text(
56                  '$_counter',
57                  style: Theme.of(context).textTheme.headlineMedium,
58                ),  // Text
59              ],  // <Widget>[]
60            ),  // Column
61          ),  // Center
62          floatingActionButton: FloatingActionButton(
63            onPressed: _incrementCounter,
64            tooltip: 'Increment',
65 +          child: const Icon(Icons.add),
66          ), // This trailing comma makes auto-formatting nicer for build methods.  // FloatingActionButt
67        );  // Scaffold
68      }
69    }
70
```

Limitations of setState()

- While setState() is a straightforward way to manage state in Flutter, it does have some limitations:

1. **Performance**: Calling setState() rebuilds the entire widget, which can be inefficient if your widget tree is large. This is because every time setState() is called, the screen is rebuilt.
2. **Global State**: setState() is not suitable for managing global app state or sharing state between distant parts of your app. It's intended for a data change in the current widget to inform the framework to repaint that widget.
3. **Coupling**: Using setState() only as a state management couples the business logic to the UI, which may reduce code re-usability and maintainability.
4. **Computation**: It is recommended that the setState() method only be used to wrap the actual changes to the state, not any computation that might be associated with the change.

Provider package as a solution

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class Counter with ChangeNotifier {
  int _count = 0;
  int get count => _count;

  void increment() {
    _count++;
    notifyListeners();
  }
}

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => Counter(),
      child: const MyApp(),
    ),  // ChangeNotifierProvider
  );
}
```

```dart
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: MyHomePage(),
    );  // MaterialApp
  }
}

class MyHomePage extends StatelessWidget {
  const MyHomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Provider Class'),
      ),  // AppBar
      body: Center(
```

The usage of const this frequently plays a vital role especially
It increases performance (i.e.: it will be known at compile time, and if there is a need to rebuild, it will simply reuse the const Widget instead of recreating it!)

Provider package as a solution

```
43        body: Center(
44          child: Column(
45            mainAxisAlignment: MainAxisAlignment.center,
46            children: <Widget>[
47              const Text(
48                'You have pushed the button this many times:',
49              ),  // Text
50              Consumer<Counter>(
51                builder: (_, counter, __) => Text(
52                  '${counter.count}',
53                  style: Theme.of(context).textTheme.headlineMedium,
54                ),  // Text
55              ),  // Consumer
56            ],  // <Widget>[]
57          ),  // Column
58        ),  // Center
59        floatingActionButton: FloatingActionButton(
60          onPressed: () => Provider.of<Counter>(context, listen: false).increment(),
61          tooltip: 'Increment',
62 +        child: const Icon(Icons.add),
63        ),  // FloatingActionButton
64      );  // Scaffold
65    }
66  }
67
```

In this example, Counter is a class that extends ChangeNotifier and includes a _count variable and an increment method.

The ChangeNotifierProvider widget is used in the main() function to create an instance of Counter and make it available to all descendant widgets.

The MyHomePage widget uses Consumer<Counter> to listen to changes in Counter and display the _count property.

The FloatingActionButton's onPressed property is set to Provider.of<Counter>(context, listen: false).increment(), which increments _count and calls notifyListeners() to trigger a rebuild.

Additional Reading: **Provider.of<Counter>(context, listen: false).increment();**

- In the context of the Provider package in Flutter, listen: false is used when you want to access the data from the provider but don't want the widget to rebuild every time the data changes.
- When you use Provider.of<Counter>(context), it's like you're "watching" the Counter object. This means that every time the Counter object changes (i.e., every time notifyListeners() is called in Counter), the widget that uses Provider.of<Counter>(context) will rebuild.
- However, when you use Provider.of<Counter>(context, listen: false), you're telling Flutter that you want to access the Counter object, but you don't care about its changes. You don't want the widget to rebuild every time the Counter changes. This is useful in certain scenarios, for example, when you want to call a method on the provider but don't need to display the updated data in the current widget.
- In your specific code snippet, Provider.of<Counter>(context, listen: false).increment();, you're accessing the Counter object to call its increment method, but you're not interested in rebuilding the widget when the Counter changes

More about Provider

- It is possible to have multiple ChangeNotifierProvider widgets in a single Flutter project. To do that we can use MultiProvder widget to provide multiple objects:

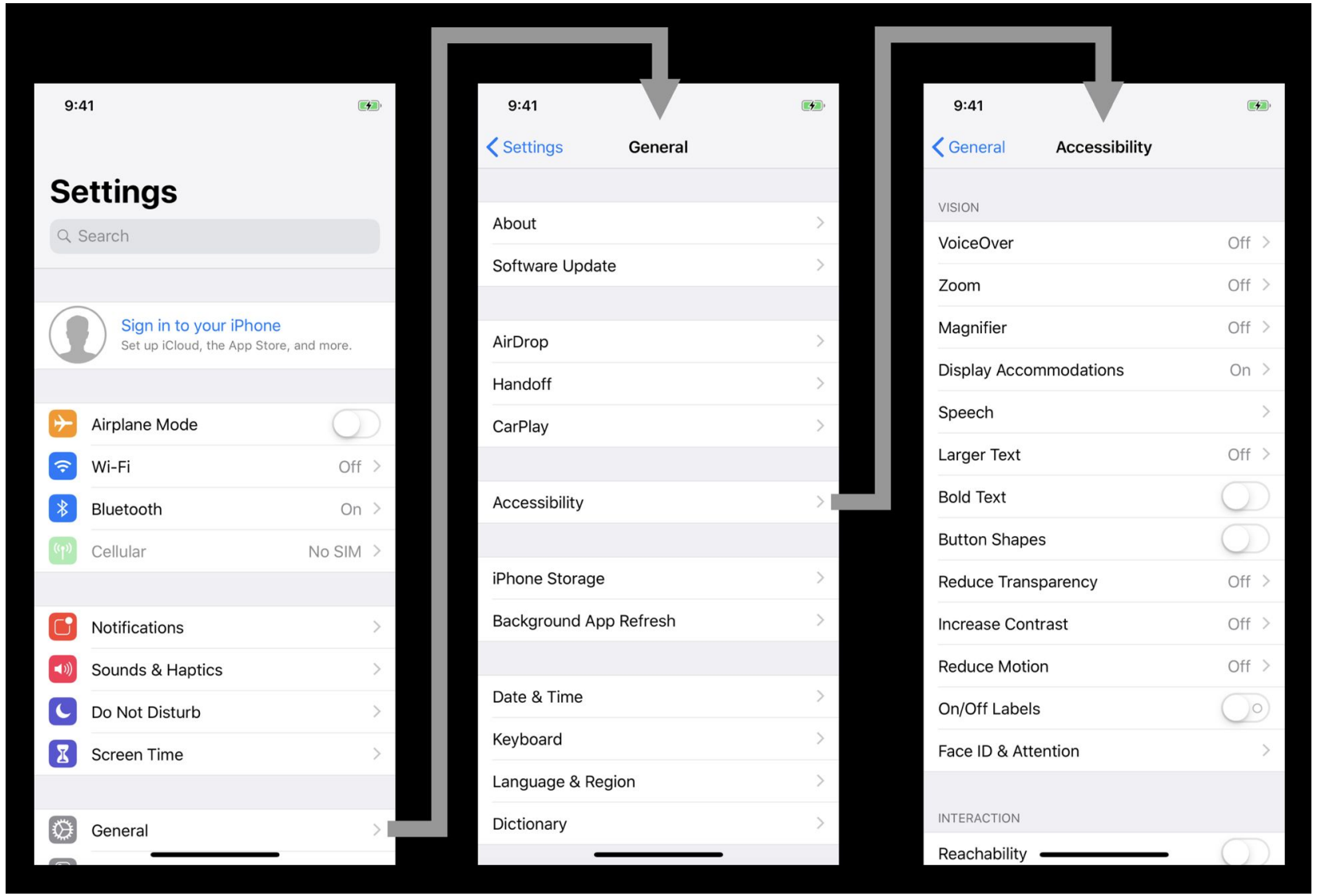
```
MultiProvider(
  providers: [
    ChangeNotifierProvider<FirstNotifier>(
      create: (_) => FirstNotifier(),
    ),
    ChangeNotifierProvider<SecondNotifier>(
      create: (_) => SecondNotifier(),
    ),
  ],
  child: YourWidget(),
)
```

In this example, FirstNotifier and SecondNotifier are classes that extend ChangeNotifier. The MultiProvider widget is used to create instances of these classes and make them available to all descendant widgets.
However, it's important to note that if you use the same type for more than one ChangeNotifierProvider, it will give you the value of the nearest provider in the widget tree. So, if you need to provide multiple objects of the same type, you should create separate classes for each object.

Introduction Navigation and Routing in Flutter

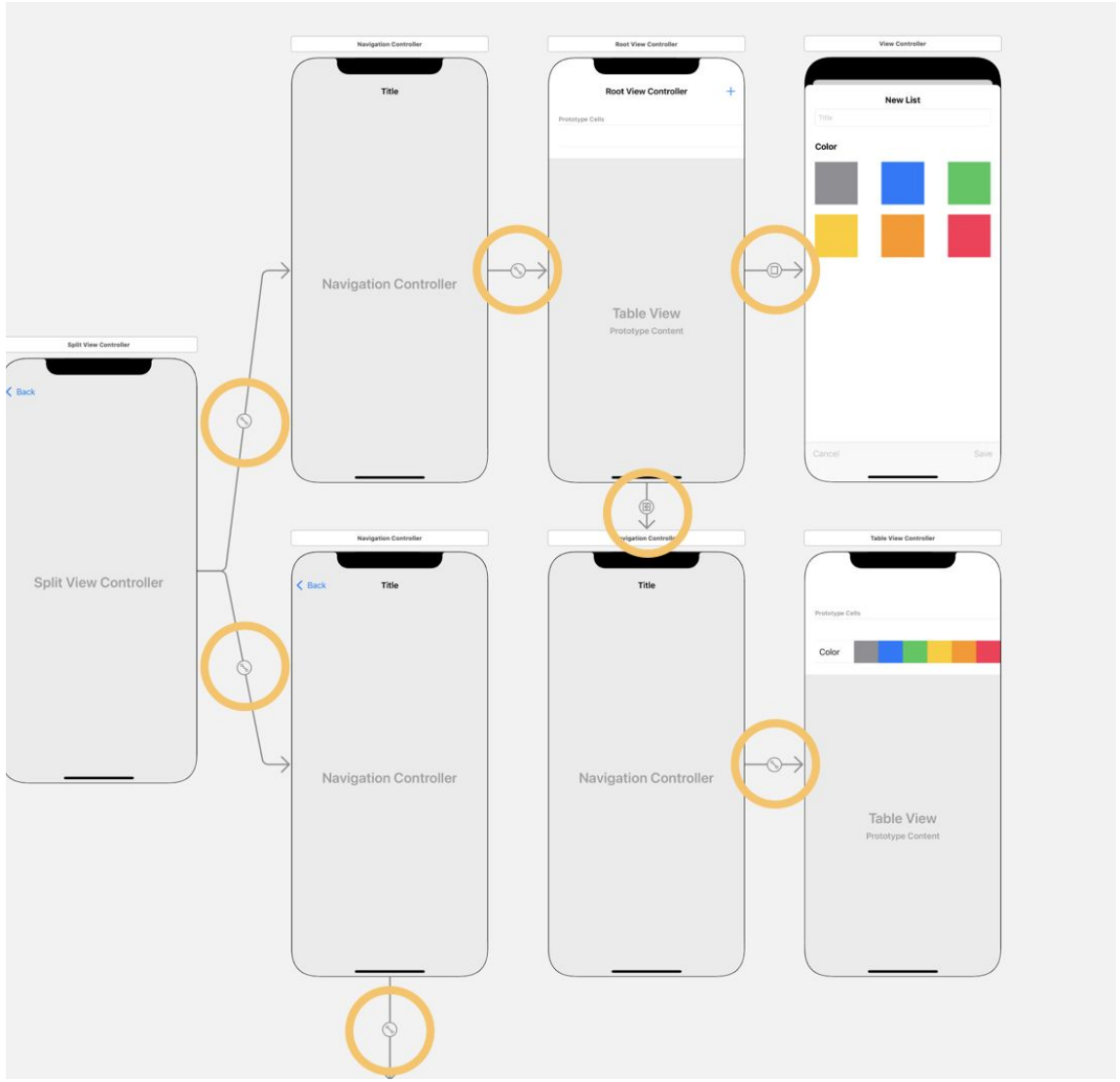
- In Flutter, navigation and routing refer to the ability to move between different screens (or routes) within an application. Here's a brief introduction:
  - **Routes**: In Flutter, a route is just a widget. The screen and pages are called a route. In Android, it is similar to an Activity, and in iOS, it is similar to ViewController.
  - **Navigator**: The Navigator widget manages the stack of routes and provides methods to navigate between them. You can use Navigator.push() to navigate to a new route and Navigator.pop() to return to the previous route.
  - **Named Routes**: Applications with simple navigation and deep linking requirements can use the Navigator for navigation and the MaterialApp.routes parameter for deep links.
  - **Router**: For applications with advanced navigation and routing requirements, Flutter provides the Router widget. This is useful for apps that use direct links to each screen, or an app with multiple Navigator widgets.

Importance of Navigation in Mobile apps

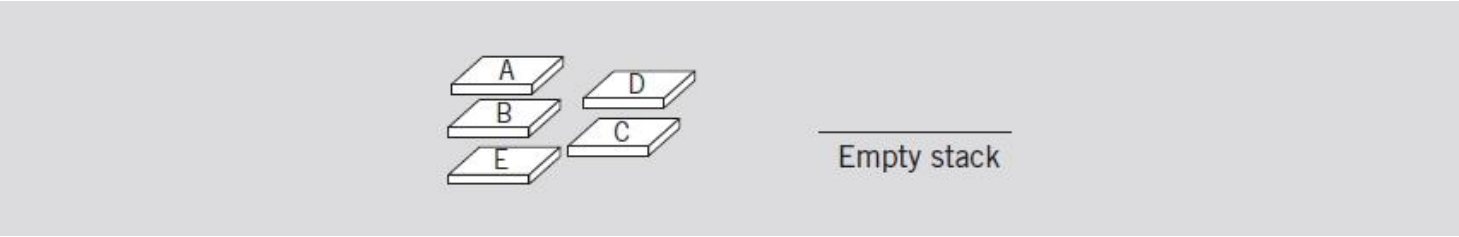


Overview of Navigation in Flutter

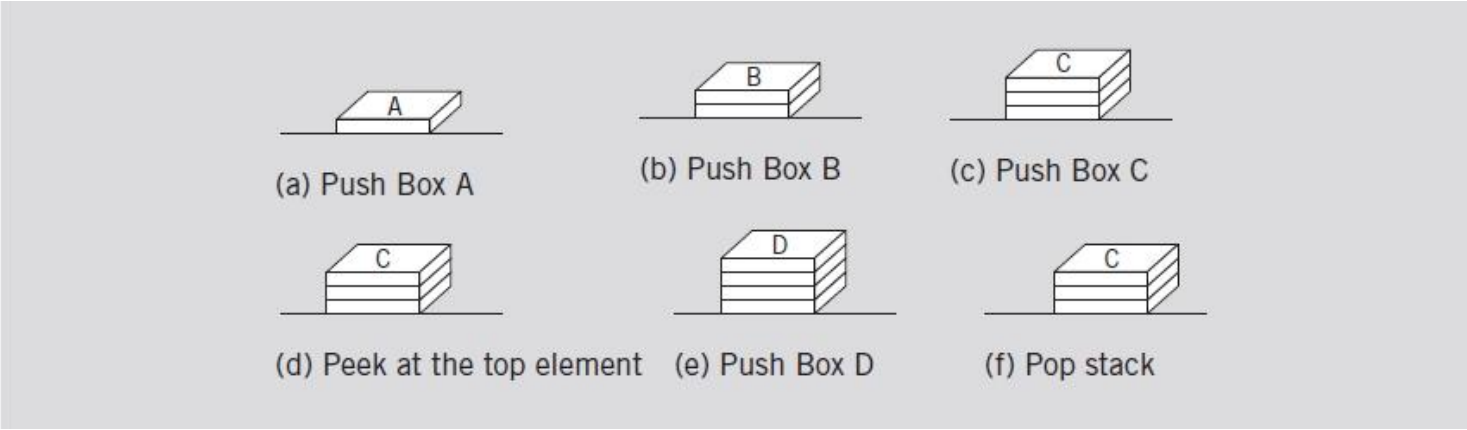
In Flutter, navigation refers to the ability to move between different screens within an application. This is managed by the Navigator widget, which handles the stack of routes and provides methods to navigate between them.



The Navigator Class



Empty stack



Stack operations

Example for Navigator.push() and Navigator.pop()

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(
    home: FirstRoute(),
  ));  // MaterialApp
}
//First Route (ViewController in iOS or Activity in Android)
class FirstRoute extends StatelessWidget {
  const FirstRoute({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('First Route'),
      ),  // AppBar
      body: Center(
        child: ElevatedButton(
          child: const Text('Open route'),
```

Example for Navigator.push() and Navigator.pop()

```
20            child: const Text('Open route'),
21            onPressed: () {
22              Navigator.push(
23                context,
24                MaterialPageRoute(builder: (context) => const SecondRoute()),
25              );
26            },
27          ),  // ElevatedButton
28        ),  // Center
29      );  // Scaffold
30    }
31  }
32  //Second Route (ViewController in iOS or Activity in Android)
33  class SecondRoute extends StatelessWidget {
34    const SecondRoute({super.key});
35
36    @override
37    Widget build(BuildContext context) {
38      return Scaffold(
39        appBar: AppBar(
40          title: const Text('Second Route'),
41        ),  // AppBar
```

Example for Navigator.push() and Navigator.pop()

```dart
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Second Route'),
      ),  // AppBar
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            Navigator.pop(context);
          },
          child: const Text('Go back!'),
        ),  // ElevatedButton
      ),  // Center
    );  // Scaffold
  }
}
```

In this example, the FirstRoute widget has a button that navigates to the SecondRoute widget when pressed. This is done using Navigator.push(). The SecondRoute widget has a button that returns to the FirstRoute widget when pressed. This is done using Navigator.pop().

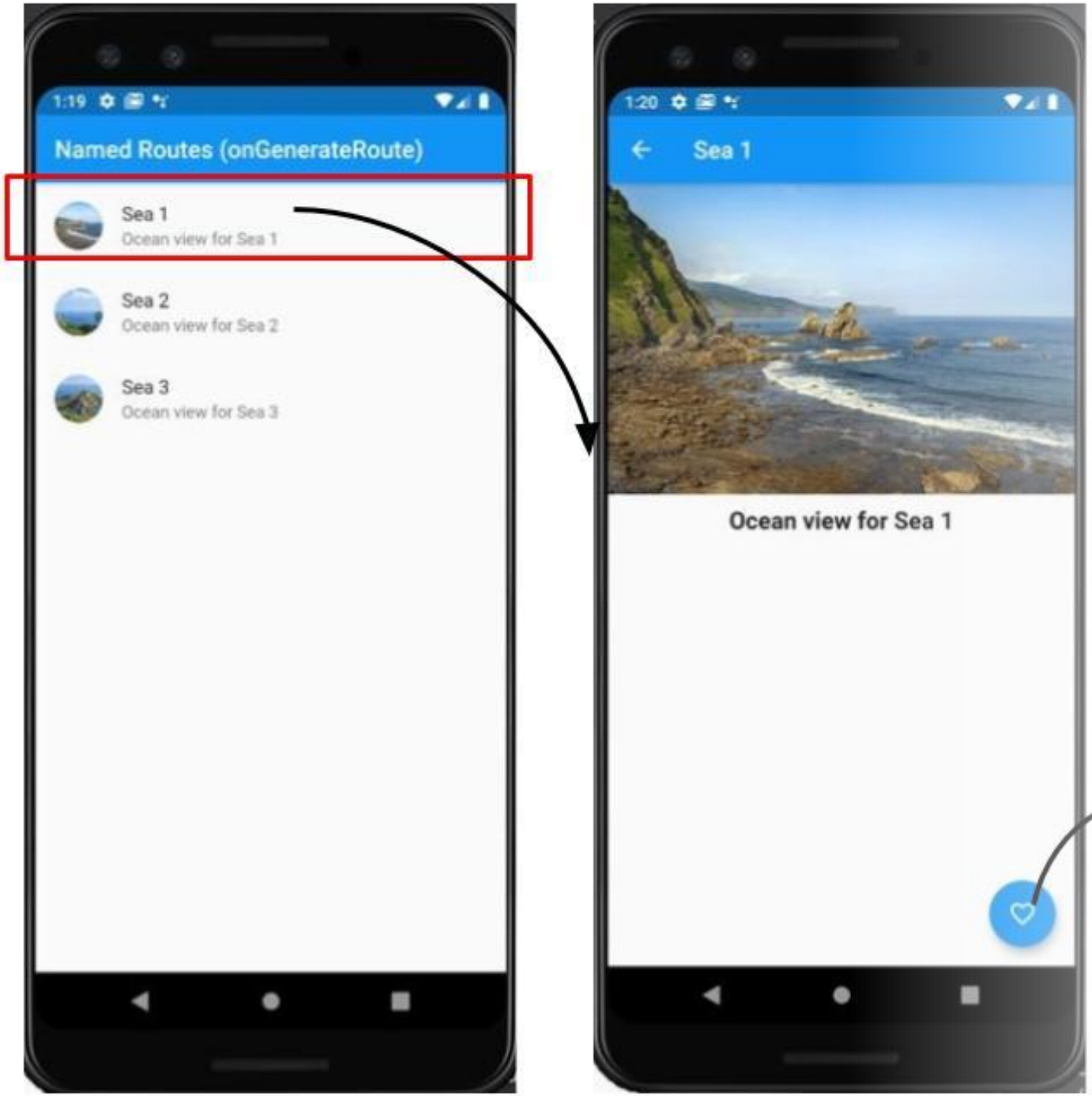
Creating small transitions between screens

```
Navigator.push(
  context,
  PageRouteBuilder(
    pageBuilder: (context, animation, secondaryAnimation) => const SecondRoute(),
    transitionsBuilder: (context, animation, secondaryAnimation, child) {
      var begin = const Offset(1.0, 0.0);
      var end = Offset.zero;
      var tween = Tween(begin: begin, end: end);
      var offsetAnimation = animation.drive(tween);

      return SlideTransition(
        position: offsetAnimation,
        child: child,
      );  // SlideTransition
    },
  ),  // PageRouteBuilder
);
```

In this example, SecondRoute is the widget that you navigate to. The SlideTransition widget is used to create a slide transition animation.

Offset is a representation of x and y coordinates.
Making Offset from (1, 0) as the beginning and (0,0) as the end means we want our animations start from the right and transition to the left.



Definition of Routes

- In Flutter, a route is a concept used to represent a screen or page in your application. Each route can be written as a separate class and has its own contents and UI. The Navigator widget manages the stack of routes and provides methods to navigate between them. You can use Navigator.push() to navigate to a new route and Navigator.pop() to return to the previous route. Applications with simple navigation and deep linking requirements can use the Navigator for navigation and the MaterialApp.routes parameter for deep links.

Navigating by Route names (Named Routes)

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(
    title: 'Named Routes Demo',
    initialRoute: '/', // the very first screen
    routes: {
      '/': (context) => const FirstScreen(),//the very first screen is set to FirstScreen Route
      '/second': (context) => const SecondScreen(),//the next /second screen is set ot SecondScreen R
    },
  ));  // MaterialApp
}

class FirstScreen extends StatelessWidget {
  const FirstScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
```

Name is assigned
At the start of the
Compilation within
runApp(); Names
Of Routes can be
Assigned within
MaterialApp() widget
Just need to set
initialRoute, and all
the routes as shown
In the example

Note: usage of '/' as the initialRoute or in the beginning of namedRoutes is a common practice and recommended. However, not required!

Navigating by Route names

```dart
18  Widget build(BuildContext context) {
19    return Scaffold(
20      appBar: AppBar(
21        title: const Text('First Screen'),
22      ),  // AppBar
23      body: Center(
24        child: ElevatedButton(
25          child: const Text('Navigate to second screen'),
26          onPressed: () {
27            Navigator.pushNamed(context, '/second');
28          },
29        ),  // ElevatedButton
30      ),  // Center
31    );  // Scaffold
32  }
33 }
34
```

Instead of using Navigator.push(), we are using Navigator.pushNamed() and
And providing the name to Which we would like to Transition.

Navigating by Route names

```dart
class SecondScreen extends StatelessWidget {
  const SecondScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Second Screen'),
      ),  // AppBar
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            Navigator.pop(context);
          },
          child: const Text('Go back to first screen!'),
        ),  // ElevatedButton
      ),  // Center
    );  // Scaffold
  }
}
```

In this example, the MaterialApp widget has a routes property that defines the available named routes and the widgets to build when navigating to those routes. The FirstScreen widget has a button that navigates to the SecondScreen widget when pressed. This is done using Navigator.pushNamed(). The SecondScreen widget has a button that returns to the FirstScreen widget when pressed. This is done using Navigator.pop().

Tab-based navigation (DefaultTabController

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const TabBarDemo());
}

class TabBarDemo extends StatelessWidget {
  const TabBarDemo({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: DefaultTabController(
        length: 3,
        child: Scaffold(
          appBar: AppBar(
```



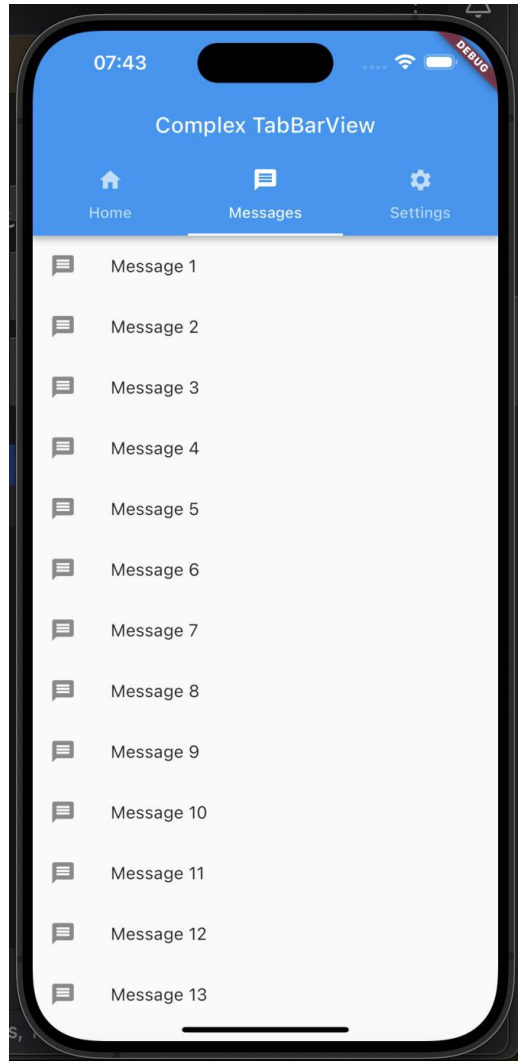
Tab-based navigation

```
17                bottom: const TabBar(
18                  tabs: [
19 🚗                Tab(icon: Icon(Icons.directions_car)),
20 🚌                Tab(icon: Icon(Icons.directions_transit)),
21 🚲                Tab(icon: Icon(Icons.directions_bike)),
22                  ],
23                ),  // TabBar
24                title: const Text('Tabs Demo'),
25              ),  // AppBar
26              body: const TabBarView(
27                children: [
28 🚗              Icon(Icons.directions_car),
29 🚌              Icon(Icons.directions_transit),
30 🚲              Icon(Icons.directions_bike),
31                ],
32              ),  // TabBarView
33            ),  // Scaffold
34          ),  // DefaultTabController
35        );  // MaterialApp
36      }
37    }
```

In this example, DefaultTabController creates a TabController and makes it available to all descendant widgets. The TabBar widget creates tabs and places them within an AppBar. The TabBarView widget displays content based on the currently selected tab.

Using DefaultTabViewController class

```
body: TabBarView(
  children: [
    const Center(child: Text('Home Page', style: TextStyle(fontSize: 24))),
    ListView.builder(
      itemCount: 20,
      itemBuilder: (context, index) {
        return ListTile(
          leading: const Icon(Icons.message),
          title: Text('Message ${index + 1}'),
        );  // ListTile
      },
    ),  // ListView.builder
    Center(
      child: ElevatedButton(
        onPressed: () {},
        child: const Text('Settings Button'),
      ),  // ElevatedButton
    ),  // Center
  ],
),  // TabBarView
```



Using TabController class

- The TabController class in Flutter coordinates tab selection between a TabBar and a TabBarView.
- The index property is the index of the selected tab and the animation represents the current scroll positions of the tab bar and the tab bar view.
- The selected tab's index can be changed with animateTo. A stateful widget that builds a TabBar or a TabBarView can create a TabController and share it directly.
- When the TabBar and TabBarView don't have a convenient stateful ancestor, a TabController can be shared by providing a DefaultTabController inherited widget.

Using TabController class

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: MyHomePage(),
    );  // MaterialApp
  }
}
```

```dart
class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key});

  @override
  _MyHomePageState createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> with SingleTickerProviderStateMixin {
  late TabController _tabController;

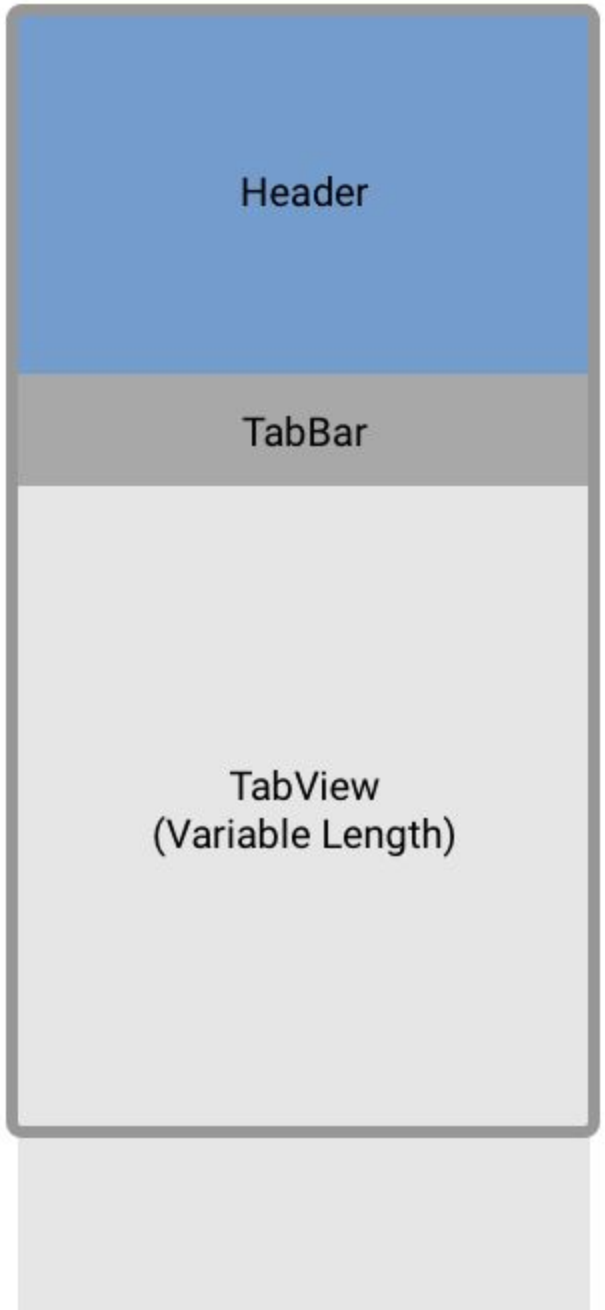
  @override
  void initState() {
    super.initState();
    _tabController = TabController(length: 2, vsync: this);
  }

  @override
  void dispose() {
    _tabController.dispose();
    super.dispose();
  }
```

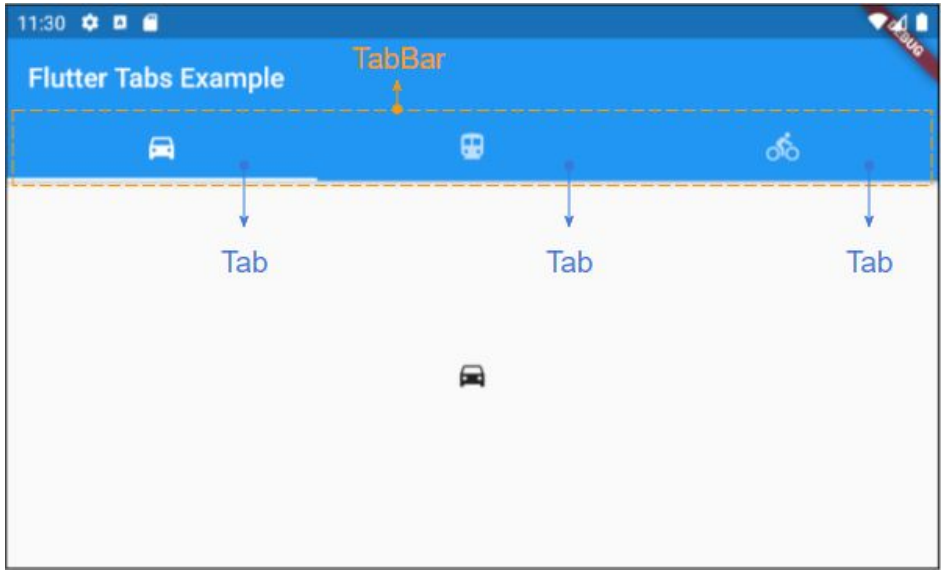
Using TabController class

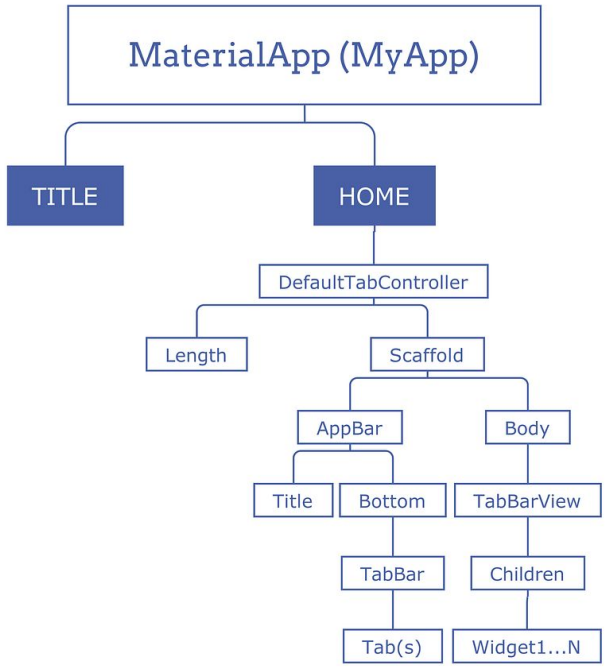
```dart
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('TabController Example'),
        bottom: TabBar(
          controller: _tabController,
          tabs: const [
            Tab(icon: Icon(Icons.home), text: 'Home'),
            Tab(icon: Icon(Icons.settings), text: 'Settings'),
          ],
        ),  // TabBar
      ),  // AppBar
      body: TabBarView(
        controller: _tabController,
        children: const [
          Center(child: Text('Home Page', style: TextStyle(fontSize: 24))),
          Center(child: Text('Settings Page', style: TextStyle(fontSize: 24))),
        ],
      ),  // TabBarView
    );  // Scaffold
  }
}
```

In this example, MyHomePage is a StatefulWidget that creates an instance of _MyHomePageState. _MyHomePageState is a class that extends State and includes a _tabController variable[1]. The initState method initializes _tabController with a TabController instance[1]. The dispose method disposes of _tabController when it is no longer needed[1]. The build method returns a widget tree that includes a Scaffold with an AppBar and a TabBarView[1].



Block
(Scrollable Section)





Important to note!

- TabController requires a StatefulWidget because it needs to create an animation controller, which is an instance of AnimationController.
- AnimationController requires a TickerProvider, which is typically provided by the vsync argument.
- In Flutter, TickerProvider is usually provided by mixing in the SingleTickerProviderStateMixin or TickerProviderStateMixin into a State object. These are only available in a StatefulWidget.
- The State object of a StatefulWidget lives for the lifetime of the widget in the tree, which means it's the perfect place to create an AnimationController.
- When the widget is removed from the tree, the State object is also disposed of, which triggers the disposal of the AnimationController.

Bottom Navigation bars




```dart
return Scaffold(
  appBar: AppBar(
    title: const Text('BottomNavigationBar Demo'),
  ),  // AppBar
  bottomNavigationBar: BottomNavigationBar(
    items: const <BottomNavigationBarItem>[
      BottomNavigationBarItem(
        icon: Icon(Icons.home),
        label: 'Home',
      ),  // BottomNavigationBarItem
      BottomNavigationBarItem(
        icon: Icon(Icons.business),
        label: 'Business',
      ),  // BottomNavigationBarItem
      BottomNavigationBarItem(
        icon: Icon(Icons.school),
        label: 'School',
      ),  // BottomNavigationBarItem
    ],  // <BottomNavigationBarItem>[]
  ),  // BottomNavigationBar
);  // Scaffold
```

In this example, BottomNavigationBar is used within a Scaffold widget. The items property is a list of BottomNavigationBarItem widgets, each representing a tab. Each BottomNavigationBarItem requires an icon and a label.

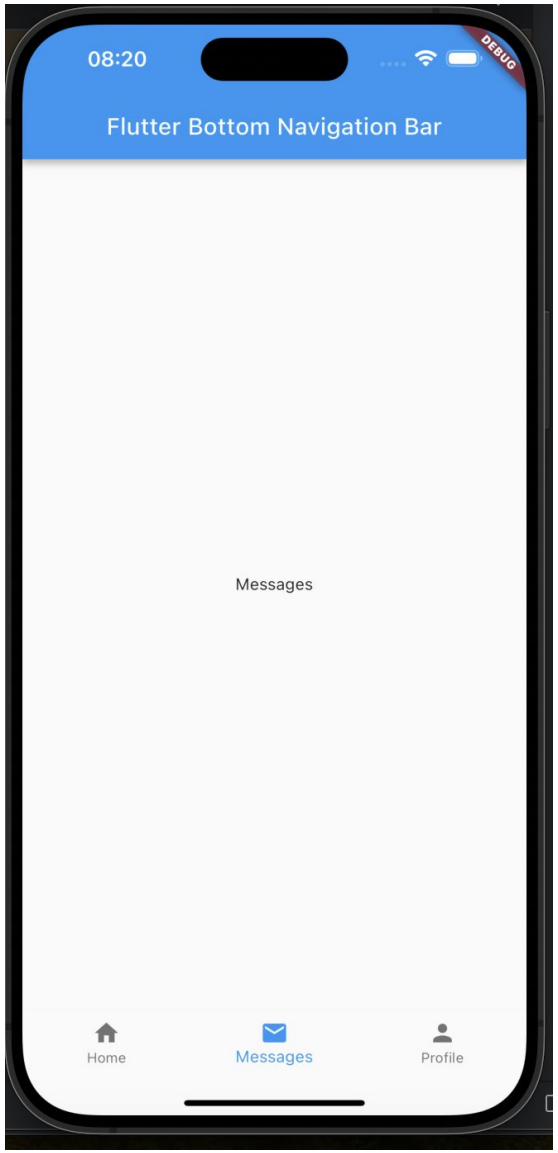
Handling interactions with the bottom navigation bar

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Bottom Navigation Bar',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),  // ThemeData
      home: const MyHomePage(),
    );  // MaterialApp
  }
}
```



Handling interactions with the bottom navigation bar

```dart
class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key});

  @override
  _MyHomePageState createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  int _currentIndex = 0;
  final List<Widget> _children = [
    const Text('Home'),
    const Text('Messages'),
    const Text('Profile')
  ];

  void onTabTapped(int index) {
    setState(() {
      _currentIndex = index;
    });
  }

```

We need to have
Our index within
The setState() widget
As we need it updated

Handling interactions with the bottom navigation bar

```dart
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Flutter Bottom Navigation Bar'),
    ),  // AppBar
    body: Center(
      child: _children[_currentIndex],
    ),  // Center
    bottomNavigationBar: BottomNavigationBar(
      onTap: onTabTapped,
      currentIndex: _currentIndex,
      items: const [
        BottomNavigationBarItem(
          icon: Icon(Icons.home),
          label: 'Home',
        ),  // BottomNavigationBarItem
        BottomNavigationBarItem(
          icon: Icon(Icons.mail),
          label: 'Messages',
        ),  // BottomNavigationBarItem
        BottomNavigationBarItem(
          icon: Icon(Icons.person),
          label: 'Profile'
        )  // BottomNavigationBarItem
      ],
    ),  // BottomNavigationBar
  );  // Scaffold
}
}
```

In this code, _currentIndex keeps track of the current selected index of the BottomNavigationBar, _children is a list of widgets to be displayed when a tab is selected, and onTabTapped is a function that updates _currentIndex with the tapped index.
The BottomNavigationBar widget is placed in the bottomNavigationBar property of the Scaffold.
Each BottomNavigationBarItem has an icon and a label.
When an item is tapped, the onTap callback is invoked, and the UI is updated with setState.

Passing Data Between Screens using Route arguments

As we have seen the transitions above, first
We are giving Routes names at the start of our app

```
void main() {
  runApp(MaterialApp(
    initialRoute: '/',
    routes: {
      '/': (context) => FirstScreen(),
      '/second': (context) => SecondScreen(),
    },
  ));
}
```

When transitioning using pushNamed(), we
Can use arguments: label to send some data

```
Navigator.pushNamed(
  context,
  '/second',
  arguments: 'Hello, Second Screen!',
);
```

Handling passed data

```
class SecondScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    final String args =
ModalRoute.of(context)!.settings.arguments as String;

    return Scaffold(
      appBar: AppBar(
        title: Text('Second Screen'),
      ),
      body: Center(
        child: Text(args),
      ),
    );
  }
}
```

In this code, ModalRoute.of(context)!.settings.arguments is used to extract the arguments. The arguments are then displayed in the center of SecondScreen.

If you have multiple arguments

Passing String and float values

```
Navigator.pushNamed(
  context,
  '/second',
  arguments: {
    'stringValue': 'Hello, Second Screen!',
    'floatValue': 123.45,
  },
);
```

Handling them using Map

```
class SecondScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    final Map arguments = ModalRoute.of(context)!.settings.arguments as Map;
    final String stringValue = arguments['stringValue'];
    final double floatValue = arguments['floatValue'];

    return Scaffold(
      appBar: AppBar(
        title: Text('Second Screen'),
      ),
      body: Center(
        child: Text('String value: $stringValue, Float value: $floatValue'),
      ),
    );
  }
}
```

If you would like to pass data while transitioning back (Navigator.pop())

```
// In the second screen
Navigator.pop(context, 'This is the data');

// In the first screen
final result = await Navigator.push(
  context,
  MaterialPageRoute(builder: (context) =>
SecondScreen()),
);

print(result);  // 'This is the data'
```

In this example, the second screen uses Navigator.pop(context, 'This is the data') to pop itself off the navigation stack and pass the string 'This is the data' back to the first screen. The first screen then prints the result.
You can replace 'This is the data' with any data you want to pass back. It can be a string, a number, an object, etc.
The await keyword is used to wait for the Navigator.push() method to return a result. This result is the data passed back from the second screen.

Nested Navigation

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: MainScreen(),
    );  // MaterialApp
  }
}

class MainScreen extends StatelessWidget {
  const MainScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Main Screen'),
      ),  // AppBar
      body: Center(
        child: ElevatedButton(
          child: const Text('Go to SubScreen'),
          onPressed: () {
            Navigator.of(context).push(MaterialPageRoute(
              builder: (context) => const SubNavigator(),
            ));  // MaterialPageRoute
          },
        ),  // ElevatedButton
      ),  // Center
    );  // Scaffold
  }
}
```

Nested Navigation

```dart
class SubNavigator extends StatelessWidget {
  const SubNavigator({super.key});

  @override
  Widget build(BuildContext context) {
    return Navigator(
      onGenerateRoute: (settings) {
        return MaterialPageRoute(
          builder: (context) => const SubScreen(),
        );  // MaterialPageRoute
      },
    );  // Navigator
  }
}
```

Nested Navigation

```dart
class SubScreen extends StatelessWidget {
  const SubScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('SubScreen'),
      ),  // AppBar
      body: Center(
        child: ElevatedButton(
          child: const Text('Go Back'),
          onPressed: () {
            Navigator.of(context).pop();
          },
        ),  // ElevatedButton
      ),  // Center
    );  // Scaffold
  }
}
```

In this example, MainScreen is the home screen of the app. When you press the button on the MainScreen, it navigates to SubNavigator which contains another Navigator widget. This Navigator widget generates routes for the subscreens. In this case, there's only one subscreen, SubScreen. When you press the button on the SubScreen, it pops the screen from the nested navigator, not the main navigator. This way, you can have a separate navigation stack inside your app.